7.4 Custom exceptions describe the problem

The caller needs to respond differently to negative and excessive calories. A named exception type expresses that difference without making the model decide how a web page should look.

Create the exception classes

Create `src/Tutorials/Class7/NegativeCaloriesException.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use InvalidArgumentException;

class NegativeCaloriesException extends InvalidArgumentException
{
}
```

Create `src/Tutorials/Class7/ExcessiveCaloriesException.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use InvalidArgumentException;

class ExcessiveCaloriesException extends InvalidArgumentException
{
}
```

Empty class bodies are enough. Each subclass inherits its constructor, message storage, and getMessage behavior. Both are also InvalidArgumentException and Exception instances. The custom type gives callers a more precise choice.

Replace `src/Tutorials/Class7/Dessert.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class Dessert extends Food
```

```
{
    private int $calories;

    public function __construct(string $name, int $calories)
    {
        parent::__construct($name);
        $this->setCalories($calories);
    }

    public function getCalories(): int
    {
        return $this->calories;
    }

    public function setCalories(int $calories): void
    {
        if ($calories < 0) {
            throw new NegativeCaloriesException('Calories cannot be negative.');
        }

        if ($calories > 5000) {
            throw new ExcessiveCaloriesException('Calories exceed the maximum of
5000.');
        }

        $this->calories = $calories;
    }

    public function __toString(): string
    {
        return "Dessert: {$this->name}, {$this->calories} calories";
    }
}
```

These exception classes share Dessert's namespace, so no import is needed inside Dessert.php. Their own files import their global parent class. Composer already knows how to find them through the existing App mapping.

Choose responses by type

Replace `public/tutorials/class7/index.php`:

```
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Dessert;
use App\Tutorials\Class7\ExcessiveCaloriesException;
use App\Tutorials\Class7\NegativeCaloriesException;

require_once __DIR__ . '/../../../vendor/autoload.php';
```

```
foreach ([-1, 0, 500, 5000, 5001] as $calories) {
    try {
        $dessert = new Dessert('Berry crumble', $calories);
        echo $dessert, '<br>', PHP_EOL;
    } catch (NegativeCaloriesException $e) {
        echo 'Correction: use zero or a positive calorie count.', '<br>', PHP_EOL;
    } catch (ExcessiveCaloriesException $e) {
        echo 'Correction: use no more than 5000 calories.', '<br>', PHP_EOL;
    }
}
```

Run the CLI script. Negative and excessive values get different correction messages; the three valid cases print desserts. There is deliberately no general Exception catch here. We know how to respond to these two domain problems; we have not invented a recovery policy for everything else.

Change the negative exception's diagnostic message temporarily and rerun. The correction response remains the same because selection uses the class, not a comparison with getMessage. Restore the original message.

The model's responsibility is to prevent an invalid value. The caller's responsibility is to decide what that failure means for its task. A browser caller may redisplay a form; a CLI caller may print a correction; a different caller may let the exception propagate. The same model supports all three.

Verify the type and state contracts

Confirm the range checks occur before assignment and that constructor and setter use the same rules. Reuse 7.2's existing-object demonstration with the new specific catch type to confirm that a rejected negative update preserves 250. Try 5001 with ExcessiveCaloriesException as well.